



INE-PYTHON

LECTURE - 09 EXCEPTIONS HANDLING

What You Will Learn

- Introduction
- Try-Except Suits
- Try-Except-Else
- Try-Except-Finally
- Try-Except-Else-Finally

EXCEPTIONS HANDLING



Introduction

EXAMPLE OF COMMON EXCEPTIONS

 `excpt.py > ...`
`1 x = 1/0`

```
[Running] python -u "/Users/anirachmingkhan/Code/INE-Python/excpt.py"
```

```
Traceback (most recent call last):
```

```
File "/Users/anirachmingkhan/Code/INE-Python/excpt.py", line 1, in  
<module>
```

```
x = 1/0
```

```
ZeroDivisionError: division by zero
```

```
[Done] exited with code=1 in 0.522 seconds
```

EXCEPTION

- **ข้อยกเว้น (Exception):** คือข้อผิดพลาดที่เกิดขึ้นขณะโปรแกรมกำลังทำงาน
 - โดยปกติจะทำให้โปรแกรมหยุดทำงานทันทีอย่างกะทันหัน
- **ข้อความติดตามข้อผิดพลาด (Traceback):** เป็นข้อความแจ้งข้อผิดพลาดที่แสดงข้อมูลเกี่ยวกับหมายเลขบรรทัดของ โปรแกรมที่ทำให้เกิดข้อยกเว้น
 - บ่งบอกประเภทของข้อยกเว้นและคำอธิบายสั้น ๆ ของข้อผิดพลาดที่ทำให้เกิดข้อยกเว้นนั้น

EXCEPTION

ข้อบกพร่องหลายอย่างสามารถป้องกันได้ด้วยการเขียนโค้ดอย่างรอบคอบตัวอย่างเช่น การตรวจสอบความถูกต้องของข้อมูลนำเข้า (input validation) โดยปกติจะเกี่ยวข้องกับการตัดสินใจอย่างง่ายในโค้ดแต่ก็มีข้อบกพร่องบางอย่างที่ไม่สามารถหลีกเลี่ยงได้ด้วยการเขียนโค้ดอย่างรอบคอบ ตัวอย่างเช่น

- การพยายามแปลงข้อความที่ไม่ใช่ตัวเลขเป็นจำนวนเต็ม
- การพยายามเปิดไฟล์เพื่ออ่านข้อมูลซึ่งไฟล์นั้นไม่มีอยู่จริง

EXCEPTION HANDLING CONCEPT

แนวคิดสำคัญเกี่ยวกับการจัดการข้อผิดพลาด:

- ข้อผิดพลาด (Exceptions): ความผิดพลาดระหว่างการทำงานของโปรแกรมที่ขัดจังหวะการทำงานปกติ
- บล็อก Try: โค้ดที่อาจก่อให้เกิดข้อผิดพลาดจะถูกวางไว้ใน try
- บล็อก Except: โค้ดที่ใช้จัดการข้อผิดพลาด หากเกิดขึ้น จะอยู่ใน except
- บล็อก Else: โค้ดที่จะทำงานหากไม่มีข้อผิดพลาดเกิดขึ้นใน try
- บล็อก Finally: โค้ดที่ทำงานเสมอ ไม่ว่าข้อผิดพลาดจะเกิดขึ้นหรือไม่ มักใช้สำหรับการทำความสะอาด เช่น ปิดไฟล์
- การจับข้อผิดพลาดหลายประเภท: ใช้หลาย except เพื่อจัดการข้อผิดพลาดต่างชนิด
- การจับข้อผิดพลาดทั้งหมด: ใช้คลาส Exception เพื่อจับข้อผิดพลาดทุกประเภท
- การโยนข้อผิดพลาด: ใช้คำสั่ง raise เพื่อสร้างข้อผิดพลาดตามเงื่อนไขที่กำหนด
- ข้อผิดพลาดที่กำหนดเอง: นิยามคลาสข้อผิดพลาดของตนเองเพื่อจัดการสถานการณ์เฉพาะ
- การจัดการทรัพยากร: ใช้ finally เพื่อให้แน่ใจว่าทรัพยากรถูกปิดหรือคืนค่า แม้เกิดข้อผิดพลาด
- ข้อความแสดงข้อผิดพลาด: การให้ข้อความที่มีประโยชน์เพื่อช่วยดีบั๊กและเข้าใจสาเหตุของข้อผิดพลาด
- แนวปฏิบัติที่ดี: เขียนโค้ดที่สะอาด ดูแลรักษาง่าย และจัดการข้อผิดพลาดได้อย่างเหมาะสม

COMMON TYPES OF EXCEPTIONS

การเข้าใจประเภทของข้อผิดพลาดที่พบบ่อยถือเป็นสิ่งสำคัญในการจัดการข้อผิดพลาดอย่างมีประสิทธิภาพใน Python ข้อผิดพลาดเหล่านี้ช่วยระบุและแก้ไขปัญหาที่อาจเกิดขึ้นระหว่างการทำงานของโปรแกรม การทราบวิธีจัดการกับข้อผิดพลาดเหล่านี้จะช่วยให้โค้ดมีความน่าเชื่อถือและยืดหยุ่นมากขึ้น ต่อไปนี้คือตัวอย่างข้อผิดพลาดที่มักพบเป็นประจำ ซึ่งนักพัฒนา Python ควรรู้จัก:

- **SyntaxError**: เกิดเมื่อมีข้อผิดพลาดด้านไวยากรณ์ของโค้ด ทำให้ไม่สามารถแปลคำสั่งได้
- **TypeError**: เกิดเมื่อใช้ฟังก์ชันหรือการดำเนินการกับชนิดข้อมูลที่ไม่เหมาะสม เช่น การบวกตัวเลขกับสตริง
- **ValueError**: เกิดเมื่อฟังก์ชันได้รับค่าอาร์กิวเมนต์ที่ชนิดข้อมูลถูกต้องแต่ค่าผิด เช่น จำนวนลบที่คาดว่าจะควรเป็นบวก
- **IndexError**: เกิดเมื่อพยายามเข้าถึงตำแหน่งในลิสต์ที่อยู่นอกขอบเขต
- **KeyError**: เกิดเมื่อใช้คีย์ที่ไม่มีอยู่ในดิกชันนารี
- **ZeroDivisionError**: เกิดเมื่อพยายามหารตัวเลขด้วยศูนย์ ซึ่งทางคณิตศาสตร์ไม่สามารถทำได้

EXCEPTIONS HANDLING



Try-Except Block

EXCEPTION

บล็อก `try` ช่วยให้คุณสามารถทดสอบโค้ดที่อาจเกิดข้อผิดพลาดได้ โดยครอบคลุมโค้ดที่อาจสร้างข้อผิดพลาดขึ้น หากเกิดข้อผิดพลาดภายในบล็อก `try` การควบคุมโปรแกรมจะถูกส่งไปยังบล็อก `except` แทนที่ ซึ่งจะมีโค้ดที่ใช้จัดการข้อผิดพลาดนั้น การใช้ `try` และ `except` ร่วมกันช่วยให้โปรแกรมไม่ล้มเมื่อเกิดข้อผิดพลาดที่ไม่คาดคิด และสามารถแสดงข้อความหรือจัดการสถานการณ์นั้นได้อย่างเหมาะสม โครงสร้างนี้มีความสำคัญอย่างยิ่งในการสร้างโปรแกรมที่มีความยืดหยุ่นและทนทาน

ไวยากรณ์:

```
1 try:
2     # code that may raise an exception
3 except ExceptionType:
4     # code to handle the exception
```

TRY-EXCEPT BLOCK

```
1 # Example of common exceptions
2 try:
3     x = 1 / 0 # ZeroDivisionError
4 except ZeroDivisionError as e:
5     print(f"Error: {e}")
6
7 print("End of program")
8 #Output : Error: division by zero
```

Listing 9.2: Example of Common Exceptions

DISPLAYING AN EXCEPTION'S DEFAULT ERROR MESSAGE

ออบเจกต์ข้อยกเว้น (Exception object): ออบเจกต์ที่ถูกสร้างขึ้นในหน่วยความจำเมื่อเกิดข้อยกเว้น โดยปกติจะมีข้อความแสดงข้อผิดพลาดเริ่มต้นที่เกี่ยวข้องกับข้อยกเว้นนั้น

- สามารถกำหนดออบเจกต์ข้อยกเว้นให้กับตัวแปรในส่วนของ `except clause` ได้
- ตัวอย่าง: `except ValueError as err:`
- สามารถส่งตัวแปรออบเจกต์ข้อยกเว้นไปยังฟังก์ชัน `print` เพื่อแสดงข้อความแสดงข้อผิดพลาดเริ่มต้นได้

TRY-EXCEPT BLOCK

```
1 # Get the name of a file.
2 filename = input('Enter a filename: ')
3 try:
4     # Open the file.
5     infile = open(filename, 'r')
6     # Read the file's contents.
7     contents = infile.read()
8     # Display the file's contents.
9     print(contents)
10    # Close the file.
11    infile.close()
12 except IOError:
13     print('An error occurred trying to read')
14     print('the file', filename)
15
16 print("End of program")
```

Listing 9.2: Try-Execpt File Oopen Example

EXCEPTIONS HANDLING



Catching Multiple Exceptions

HANDLING MULTIPLE EXCEPTIONS

บ่อยครั้งที่โค้ดในบล็อก `try` อาจทำให้เกิดข้อยกเว้นหลายประเภทได้จึงจำเป็นต้องเขียน `except` clause สำหรับข้อยกเว้นแต่ละประเภทที่ต้องการจัดการ

- `except` clause ที่ไม่ได้ระบุประเภทข้อยกเว้นเฉพาะ จะจัดการกับข้อยกเว้นใดๆ ที่เกิดขึ้นในบล็อก `try`
- และ `except` clause นี้ควรอยู่เป็นบรรทัดสุดท้ายในชุดของ `except` clauses

CATCHING MULTIPLE EXCEPTIONS

```
1 try:
2     value = int(input("Enter a number: "))
3     result = 10 / value
4 except ValueError:
5     print("Invalid input! Please enter a number.")
6 except ZeroDivisionError:
7     print("Cannot divide by zero!")
8
9 print("End of program")
```

Listing 9.3: Catching Multiple Exceptions

EXCEPTIONS HANDLING



Catching All Exceptions

EXCEPTION

- ถ้าประโยค if ในบล็อก try ทำให้เกิดข้อยกเว้น
 - ข้อยกเว้นที่ระบุใน except clause:
 - โปรแกรมจะทำงานใน handler ที่อยู่หลัง except clause นั้นทันที
 - แล้ว โปรแกรมจะทำงานต่อหลังจากบล็อก try/except
 - กรณีข้อยกเว้นอื่น ๆ ที่ไม่ตรงกับ except clause:
 - โปรแกรมจะหยุดทำงานและแสดงข้อความ traceback (traceback)
- ถ้าไม่มีข้อยกเว้นเกิดขึ้น handler จะถูกข้ามไป

CATCHING ALL EXCEPTIONS

```
1 try:
2     value = int(input("Enter a number: "))
3     result = 10 / value
4 except Exception as e:
5     print(f"An error occurred: {e}")
6
7 print("End of program")
```

Listing 9.5: Catching All Exceptions

EXCEPTIONS HANDLING



Else Block

THE ELSE CLAUSE

คำสั่ง try/except อาจมี else clause ที่เป็นทางเลือกเสริมซึ่งจะอยู่หลัง except clauses ทั้งหมด

- โดยจะจัดชิดแนวเดียวกับคำสั่ง try และ except
- ไวยากรณ์คล้ายกับ else clause ในโครงสร้างตัดสินใจ
- **บล็อก else (else suite)** คือชุดคำสั่งที่จะทำงานหลังจากคำสั่งใน try ทำงานเสร็จ โดยจะทำงานเฉพาะเมื่อไม่มีข้อยกเว้นใดๆ เกิดขึ้น
 - ถ้ามีข้อยกเว้นเกิดขึ้น else suite จะถูกข้ามไป

ELSE BLOCK

Syntax:

```
1 try:
2     # code that may raise an exception
3 except ExceptionType:
4     # code to handle the exception
5 else:
6     # code to execute if no exceptions were raised
```

ELSE BLOCK

```
1 try:
2     value = int(input("Enter a number: "))
3     result = 10 / value
4 except ZeroDivisionError:
5     print("Cannot divide by zero!")
6 else:
7     print(f"The result is {result}")
8
9 print("End of program")
```

Listing 9.5: Else Block

ELSE BLOCK

```
1 def divide (a, b):  
2     return a / b  
3  
4 a, b = map(int, input().split())  
5 print(divide(a, b))  
6 print("End of program")
```

Listing 9.6: Normal Divider function

```
1 def divide(a,b):  
2     try:  
3         result = a/b  
4     except ZeroDivisionError as e:  
5         print("Exception: ", e)  
6     else:  
7         return result  
8  
9 a,b = map(int, input().split())  
10 print(divide(a,b))  
11 print("End of program")
```

Listing 9.7: Divider function with Exception Handler

EXCEPTIONS HANDLING



Finally Block

THE FINALLY CLAUSE

คำสั่ง try/except อาจมี finally clause เป็นตัวเลือกเสริม ซึ่งจะอยู่หลัง except clauses ทั้งหมด

- โดยจัดแนวเดียวกับ try และ except

- รูปแบบทั่วไป:

finally:

คำสั่งต่างๆ

- **บล็อก finally (finally suite)** คือชุดคำสั่งที่อยู่หลังคำสั่ง finally
 - จะทำงานไม่ว่าจะเกิดข้อยกเว้นหรือไม่ก็ตาม
 - จุดประสงค์คือเพื่อทำความสะอาดหรือจัดการขั้นตอนสุดท้ายก่อนออกจากโปรแกรม

FINALLY BLOCK

Syntax:

```
1 try:
2     # code that may raise an exception
3 except ExceptionType:
4     # code to handle the exception
5 finally:
6     # code to execute regardless of exceptions
```

FINALLY BLOCK

```
1 try:
2     # Ask user to input two numbers
3     numerator = float(input("Enter the numerator: "))
4     denominator = float(input("Enter the denominator: "))
5
6     # Perform the division
7     result = numerator / denominator
8     print(f"The result is: {result}")
9
10 except ZeroDivisionError:
11     # Handle division by zero error
12     print("Error: You cannot divide by zero.")
13
14 except ValueError:
15     # Handle input error (if input is not a valid number)
16     print("Error: Invalid input. Please enter numeric values.")
17
18 finally:
19     # This block will always run, regardless of errors
20     print("Execution completed, whether an exception occurred or not.")
21
22 print("End of program")
```

Listing 9.6: Finally Block Example

EXCEPTIONS HANDLING



Combining Try-Except-Else-Finally

COMBINING TRY-EXCEPT-ELSE-FINALLY

Syntax:

```
1 try:
2     # code that may raise an exception
3 except ExceptionType:
4     # code to handle the exception
5 else:
6     # code to execute if no exceptions were raised
7 finally:
8     # code to execute regardless of exceptions
```

COMBINING TRY-EXCEPT-ELSE-FINALLY

```
1 try:
2     value = int(input("Enter a number: "))
3     result = 10 / value
4 except ValueError:
5     print("Invalid input! Please enter a number.")
6 except ZeroDivisionError:
7     print("Cannot divide by zero!")
8 else:
9     print(f"The result is {result}")
10 finally:
11     print("Execution completed.")
```

Listing 9.7: Combining Try-Except-Else-Finally

EXCEPTIONS HANDLING



Custom Exceptions

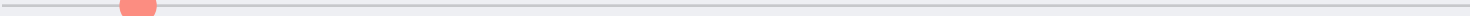
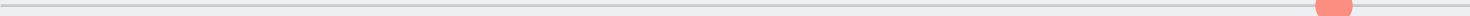
CUSTOM EXCEPTION

```
1 # Define a custom exception
2 class NegativeNumberError(Exception):
3     def __init__(self, value):
4         self.value = value
5         super().__init__(f"Invalid input: {value} is a negative number")
6
7 # Function to check for positive numbers
8 def check_positive_number(num):
9     if num < 0:
10         raise NegativeNumberError(num)
11     else:
12         print(f"{num} is a valid positive number.")
13
14 # Using the function and handling the custom exception
15 try:
16     number = int(input("Enter a positive number: "))
17     check_positive_number(number)
18 except NegativeNumberError as e:
19     print(e)
20 except ValueError:
21     print("Error: Please enter a valid integer.")
22 finally:
23     print("Program execution finished.")
```

Listing 9.11: Custom Exception Example

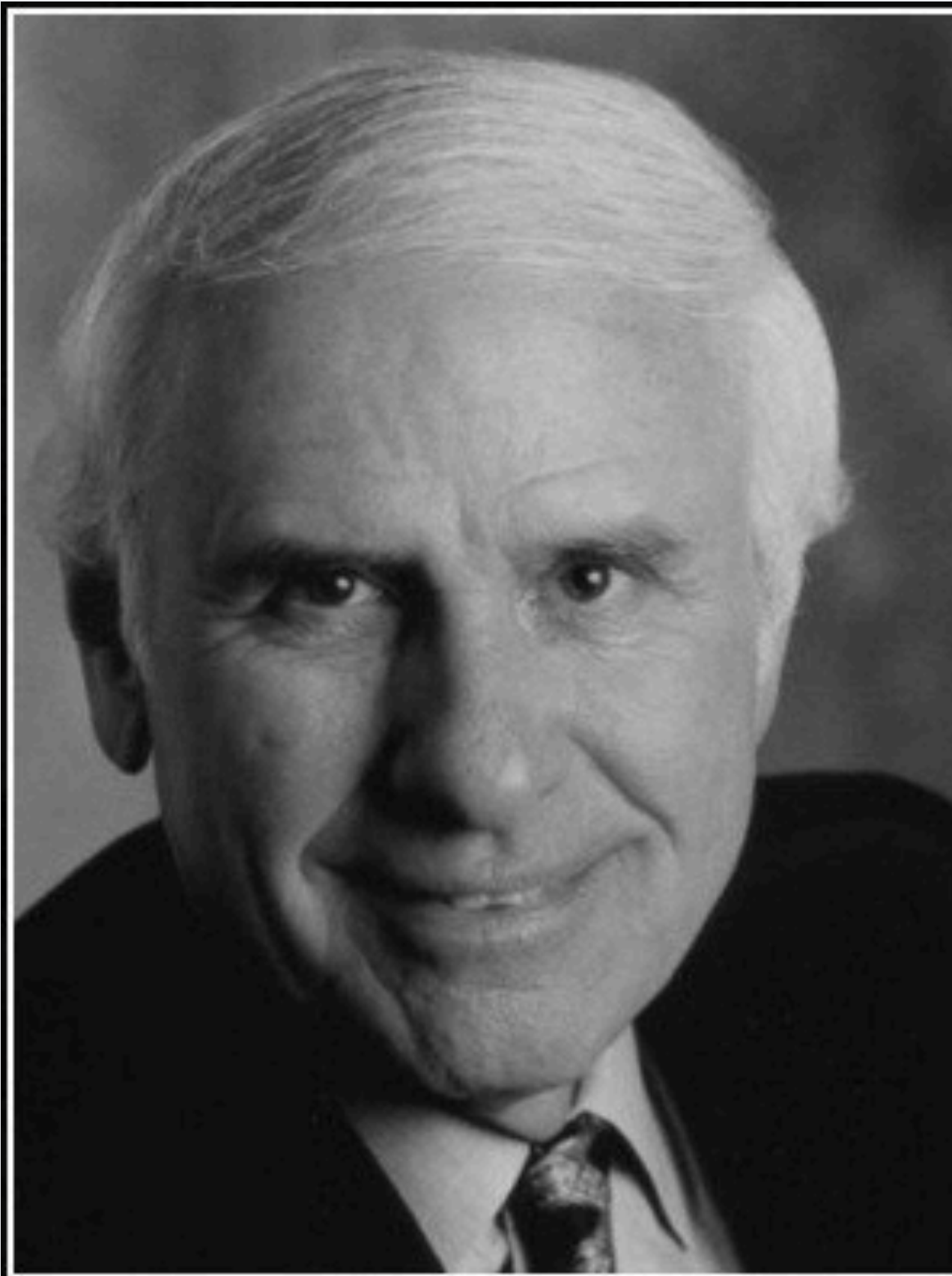
WHAT IF AN EXCEPTION IS NOT HANDLED?

- มีสองกรณีที่ข้อยกเว้นจะไม่ถูกจัดการ (unhandled) คือ
 - ไม่มี except clause ที่ระบุชนิดของข้อยกเว้นที่ถูกต้อง
 - ข้อยกเว้นเกิดขึ้นนอกบล็อก try
- ในทั้งสองกรณี ข้อยกเว้นจะทำให้โปรแกรมหยุดทำงานทันที



THANKS

WORD OF THE WISE



Don't join an easy crowd; you won't grow. Go where the expectations and the demands to perform are high.

— *Jim Rohn* —

AZ QUOTES